// 🦁 Zoo Animal Tracker — Student Task Template

// Goal: Practice OOP concepts like inheritance, abstract classes, and interfaces in Kotlin.

// You'll build a mini zoo system that tracks different animals and their unique sounds or behaviors.

// Step-by-step instructions:

// 1️⃣ Create an abstract class called "Animal" with:

// - A property: name (String)

// - An abstract function: makeSound()

// 2️⃣ Create three subclasses: "Lion", "Elephant", and "Parrot".

// - Each should override makeSound() and print a unique sound message (e.g., "Roar!", "Trumpet!", "Chirp!").

// - Add one extra method or property for each (e.g., Lion can hunt, Elephant sprays water).

// 3️⃣ Create an interface "Feedable" with a function feed(food: String).

// - Implement it in at least one subclass with a message like "Lion eats meat."

// 4️⃣ Write a function called "displayZoo(animals: List<Animal>)" that loops through animals and calls makeSound().

// 5️⃣ In main(), create a list of animals and feed some of them.

// - Use polymorphism to call their behaviors and sounds dynamically.

// 💡 Hint: Use abstract classes for shared structure and interfaces for optional behaviors.

```
// 📜 Sample Output (Example)
// --- Zoo Tracker ---
// Lion: Roar!
// Elephant: Trumpet!
// Parrot: Chirp!
// Lion eats meat.
// Elephant sprays water from trunk.

fun main() {
   // TODO: Define Animal abstract class
   // TODO: Define subclasses (Lion, Elephant, Parrot)
   // TODO: Define Feedable interface
   // TODO: Implement displayZoo() function
   // TODO: In main(), create sample animal list and call displayZoo()
}
```

```
abstract class Animal(val name: String) {
   abstract fun makeSound()
}

interface Feedable {
   fun feed(food: String)
}

class Lion(name: String) : Animal(name), Feedable {

   override fun makeSound() {
      println("$name: Roar!")
   }

   fun hunt() {
      println("$name is hunting in the jungle.")
   }

   override fun feed(food: String) {
      println("$name eats $food.")
   }
}

class Elephant(name: String) : Animal(name), Feedable {

   override fun makeSound() {
      println("$name: Trumpet!")
   }
```

```
    fun sprayWater() {
        println("$name sprays water from trunk.")
    }

    override fun feed(food: String) {
        println("$name eats $food.")
    }
}

class Parrot(name: String) : Animal(name) {

    override fun makeSound() {
        println("$name: Chirp!")
    }

    fun fly() {
        println("$name is flying around.")
    }
}

fun displayZoo(animals: List<Animal>) {
    println("--- Zoo Tracker ---")
    for (animal in animals) {
        animal.makeSound()
    }
}
```

```
fun main() {

    val lion = Lion("Lion")
    val elephant = Elephant("Elephant")
    val parrot = Parrot("Parrot")

    val zooAnimals: List<Animal> = listOf(lion, elephant, parrot)

    displayZoo(zooAnimals)

    lion.feed("meat")
    elephant.feed("grass")

    lion.hunt()
    elephant.sprayWater()
    parrot.fly()
}
```